CHAPTER 7

Load Balancing & Reverse / Forward Proxies

Part II · Building Blocks — the traffic director that makes “add more servers” work.

LEARNING OBJECTIVES

- Explain what a load balancer is and the problems it solves.
- Compare the common load-balancing algorithms and know when to use each.
- Explain how **health checks** keep traffic away from dead servers and provide high availability.
- Distinguish **Layer 4** from **Layer 7** load balancing and their trade-offs.
- Distinguish a **reverse proxy** from a **forward proxy**, with real examples.
- Explain how to make the load balancer itself highly available, including across regions.

REAL-WORLD ANALOGY — THE RECEPTIONIST AT A LARGE OFFICE

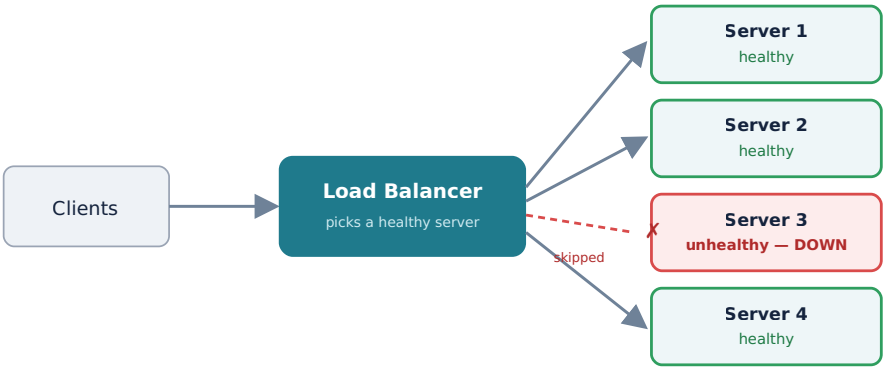
Picture a busy office with many staff members who can each help a visitor. A **receptionist** greets everyone at the door and sends each visitor to a staff member who is *free and present* — never to someone who called in sick that day (a **health check**). Visitors don’t need to know the office layout or who’s available; they just talk to the receptionist, who hides all of that and represents the whole office (a **reverse proxy**). A load balancer is exactly this receptionist for your servers.

7.1 The problem load balancing solves

In Chapter 6 we made the app tier **stateless** so we could run many identical app servers. But that raises an immediate question: when a request arrives, **which** server handles it? Without an answer, three things go wrong: clients would have to know every server’s address, one server could be swamped while others sit idle, and a crashed server would keep receiving requests that fail. A **load balancer** solves all three.

7.2 What a load balancer is

A load balancer sits between clients and your pool of servers. Clients see just one address (a virtual IP or DNS name); the load balancer spreads incoming requests across the healthy servers behind it and quietly removes any that fail. It is the component that turns “add more servers” from a wish into a working strategy — delivering both **scalability** (share the load) and **high availability** (route around failures).



Health checks let the load balancer detect Server 3 is down and route around it — the app tier keeps working despite the failure.

7.3 How it chooses a server: the algorithms

Algorithm	How it works	Best when
Round robin	Rotate through servers in order: 1, 2, 3, 1, 2, 3…	Servers are equal and requests are similar in cost.
Weighted round robin	Round robin, but bigger servers get a larger share.	Servers have different capacities.
Least connections	Send to the server with the fewest active connections.	Request durations vary a lot (some slow, some fast).
Least response time	Favor the server currently responding fastest.	You want to chase real-time performance.
IP hash	Hash the client’s IP to always pick the same server.	You need a client to stick to one server (affinity) without stored state.
Power of two choices	Pick two servers at random, send to the less loaded one.	You want near-optimal balancing with almost no coordination.

KEY INSIGHT — MATCH THE ALGORITHM TO THE WORKLOAD

Round robin is the sensible default, but it assumes every request costs the same. When some requests are far heavier than others, round robin can pile slow requests onto one unlucky server while others idle. **Least connections** adapts to that automatically. The algorithm is a small choice with a big effect on tail latency (Chapter 5) — pick it based on how uniform your requests really are.

7.4 Health checks: knowing who’s alive

A load balancer is only as good as its knowledge of which servers are healthy. Two mechanisms:

- **Active health checks:** the load balancer periodically probes each server (e.g. an HTTP `GET /health` every few seconds). If a server fails to respond correctly, it’s pulled from rotation until it recovers.

- **Passive health checks:** the load balancer watches real traffic; if a server starts erroring or timing out, it's marked unhealthy without a separate probe.

> **KEY INSIGHT — HEALTH CHECKS ARE WHAT DELIVER HIGH AVAILABILITY**
>
> Multiple servers alone don't give you high availability — you also need something to *notice* failures and route around them. Health checks are that something. Combined with a pool of stateless servers, they mean a single crashed app server is invisible to users. (The catch: the load balancer itself is now a critical component — Section 7.7.)

7.5 Layer 4 vs Layer 7 load balancing

Recall the network layers from Chapter 3. A load balancer can operate at two of them:

	Layer 4 (Transport)	Layer 7 (Application)
Works on	TCP/UDP — IP addresses and ports.	HTTP — URLs, headers, cookies.
Can it read the request?	No — it just forwards packets.	Yes — it understands the content.
Abilities	Fast, simple, protocol-agnostic passthrough.	Route by URL path or header, terminate TLS, sticky sessions by cookie, content-based routing.
Cost	Very low overhead.	Heavier — must parse (and often decrypt) each request.

So **L4 gives speed and simplicity**; **L7 gives intelligence**. For example, an L7 balancer can send `/api/*` to your API servers and `/images/*` to image servers, or route beta users by a header — things L4 simply cannot see. Most modern edges use L7 (which overlaps with the API gateway of Chapter 9); L4 is chosen when raw throughput or non-HTTP traffic (like a database connection) is the priority.

7.6 Reverse proxy vs forward proxy

A **proxy** is any intermediary that forwards traffic on someone's behalf. The chapter's two kinds differ in *whose* behalf:

```
FORWARD PROXY  (represents the CLIENT; sits on the client side)

  [ client ] \
  [ client ]  --> ( FORWARD PROXY ) --> internet --> server
  [ client ] /      hides & controls the clients
  uses: company web filter, monitoring, caching, anonymity

REVERSE PROXY  (represents the SERVER; sits on the server side)

                                    --> [ server ]
  client --> ( REVERSE PROXY ) -->  --> [ server ]
             fronts & hides servers --> [ server ]
  does: TLS termination, caching, compression, LOAD BALANCING
  examples: Nginx, HAProxy, Envoy  (a load balancer IS a reverse proxy)
```

KEY INSIGHT — THE MNEMONIC

A **forward** proxy stands in front of and represents the **client**; a **reverse** proxy stands in front of and represents the **server**. Your load balancer and API gateway are reverse proxies — they hide your servers, terminate TLS, and centralize cross-cutting concerns. A corporate internet filter is a forward proxy — it hides and controls the clients.

7.7 Making the load balancer itself reliable

We removed the single point of failure at the app tier — but now **all traffic flows through the load balancer**, so if it dies, everything is unreachable. You must make the balancer itself highly available:

- **Run more than one** load balancer — a redundant pair in active–passive (one on standby) or active–active (both serving).
- Use a **floating / virtual IP** or DNS failover so traffic shifts to the healthy balancer automatically if one fails.
- **Managed cloud load balancers** (AWS ELB/ALB/NLB, GCP, Azure) are redundant and self-healing by default — usually the simplest correct choice.

KEY INSIGHT — BALANCING HAPPENS AT MANY LEVELS

Load balancing isn't one box; it's a pattern that repeats. **Global** load balancing uses **GeoDNS** (Chapter 4) to send each user to their nearest healthy *region*; a **regional** load balancer then spreads traffic across app servers there; and you'll balance again across **database read replicas** (Part III) and between microservices (a service mesh, Chapter 30). Requests are balanced repeatedly on their way in.

Common mistakes

WATCH OUT

- Running with **no health checks** — happily sending traffic to dead servers.
- Deploying a **single** load balancer — recreating the single point of failure you tried to remove.
- Using **sticky sessions** unnecessarily (Chapter 6), which prevents even distribution.
- Choosing **round robin** when request costs vary wildly — use least connections instead.
- Terminating TLS at the balancer but leaving the balancer→backend hop unsecured in a zero-trust network.
- Confusing reverse and forward proxies, or assuming L7 features are free — they cost CPU (parsing and decryption).

Interview questions

1. What is a load balancer, and what problems does it solve?
2. Compare round robin, least connections, and IP hash. When would you choose each?
3. What is the difference between active and passive health checks?

4. Compare Layer 4 and Layer 7 load balancing. What can L7 do that L4 cannot, and at what cost?
5. Define reverse proxy and forward proxy, and give an example of each.
6. The load balancer is a single point of failure. How do you make it highly available?
7. How does global, multi-region load balancing work?
8. Why do sticky sessions complicate load balancing, and how do you avoid needing them?

Hands-on exercises

1. With 3 servers and 6 sequential requests, write out which server round robin selects each time.
2. Servers have active connection counts [5, 2, 8]. Which does “least connections” pick?
3. Decide L4 or L7 for each: routing `/api` vs `/static`; plain TCP passthrough to a database; routing by a request header.
4. Classify as reverse or forward proxy: Nginx in front of app servers; a corporate web filter; a CDN edge; a company’s outbound internet proxy.
5. Sketch a highly-available load-balancer setup using two nodes and a floating IP.

Mini project

PUT A LOAD BALANCER IN FRONT OF REAL SERVERS

Run two or three tiny web servers on different ports, each returning which port it is. Put **Nginx** (or HAProxy) in front as a reverse proxy load-balancing across them with round robin and a health check. Send repeated requests and watch the responses rotate between backends. Then **kill one backend** and confirm the load balancer stops sending it traffic while the others keep serving. You’ll have built, with your own hands, the exact behavior in this chapter’s hero diagram.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE BALANCING LAYER

Extend your Chapter 6 reference diagram with a real load-balancing layer. Choose an **algorithm** and justify it from your workload. Add **health checks**. Decide **L4 vs L7** (likely L7 at the edge, for path-based routing and TLS termination). Plan **high availability** for the balancer itself (redundant nodes or a managed cloud LB) and **global routing** (GeoDNS → regional load balancers) to match the multi-region ambitions in your estimates. Note where you’ll *also* balance across database read replicas in Part III.

Chapter summary

- A **load balancer** distributes requests across a pool of servers, giving both scalability and high availability, and presenting clients a single address.
- It chooses a server via an **algorithm** (round robin, least connections, IP hash…) — match it to how uniform your requests are.

- **Health checks** (active or passive) detect failed servers and route around them — the mechanism that actually delivers high availability.
- **Layer 4** balancing is fast, simple packet forwarding; **Layer 7** understands HTTP and can route by content and terminate TLS, at higher cost.
- A **reverse proxy** fronts servers (load balancers, Nginx, Envoy); a **forward proxy** fronts clients (corporate filters). Remember: reverse = server side, forward = client side.
- The balancer itself must be made **highly available** (redundant nodes, floating IP, or managed cloud LB), and balancing repeats at global, regional, and data-tier levels.

COMING NEXT

Chapter 8 — Caching: CDN, Redis, Memcached. We zoom into the “check cache first” box from Chapter 6 — the single highest-impact way to make a system fast. Where to cache, what to cache, how to keep caches fresh (invalidation), and the famous problems of stale data, thundering herds, and cache stampedes.